



Campus Guanajuato

División de Ciencias
Económico Administrativas

Nombre completo del alumno:

Fernando Alejandro Guevara Rodriguez

NUA:283803

Nombre de la materia:

Computo Avanzado

Nombre completo del docente asesor de la materia:

Marco Aurelio Sotelo Figueroa

Número y tema de la actividad:

Reporte explicando el ACO y como se implementó.

Fecha:

04 de diciembre del 2025

Implementación del algoritmo de Optimización por Colonia de Hormigas (ACO) para un problema del viajante (TSP) simétrico

Instrucciones:

Realizar un reporte de explicando el ACO y como se implementó.

Definiciones:

Optimización por Colonia de Hormigas (ACO)

Es un algoritmo de optimización usado para encontrar el camino más corto entre puntos o nodos.

Se desarrolla observando el comportamiento de las hormigas cuando buscan comida: las hormigas son esencialmente ciegas, por lo que siguen los rastros de feromonas que otras hormigas dejan en el camino.

El algoritmo replica este comportamiento utilizando **probabilidades** para decidir el siguiente nodo, basadas en **la distancia** y **la cantidad de feromonas** depositadas en cada posible ruta.

Problema del viajante (TSP) simétrico

Dado un conjunto de n nodos y las distancias entre cada par de nodos, el objetivo es encontrar un recorrido cerrado de **longitud mínima**, visitando cada nodo exactamente una vez. En la versión simétrica, la distancia desde el nodo i al nodo j es la misma que desde j a i .

Introducción:

Este algoritmo se desarrolla cumpliendo los siguientes requisitos:

Python

Numpy

Matplotlib

Los datos utilizados provienen de:

El problema de 100 ciudades "kroA100.tsp" de Krolak/Felts/Nelson

El propósito es encontrar la ruta más corta que recorra todos los puntos (nodos) en un camino cerrado y calcular los tiempos y compararlos los algoritmos usando paralelización por medio de gráficos de cajas.

Desarrollo:

1. Inicializar hormigas

Primero se define el número de hormigas y se colocan en posiciones iniciales dentro del espacio del TSP.

También se inicializan las feromonas en todas las aristas con un valor τ_0 y se fijan los parámetros del algoritmo (α , β , ρ , ξ).

2. Hormigas se mueven por probabilidad

Cada hormiga construye un recorrido cerrado visitando todos los nodos sin repetir. Para decidir el siguiente nodo j desde el nodo i , se usa la regla de probabilidad:

$$P_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}]^\beta}{\sum_{l \in allowed_k} [\tau_{il}(t)]^\alpha [\eta_{il}]^\beta}$$

donde $\eta_{ij} = 1/d_{ij}$ es la visibilidad heurística.

α y β controlan cuánto influye la feromona o la heurística al seleccionar el próximo nodo.

3. Feromona local se actualiza.

Cada vez que una hormiga recorre una arista $i \rightarrow j$, aplica una actualización local:

$$\tau_{ij} \leftarrow (1 - \xi) \tau_{ij} + \xi \tau_0$$

ξ : tasa de evaporación local (por ejemplo, un valor pequeño como 0.01–0.1).

τ_0 : valor inicial de feromona (p. ej. basado en una solución heurística inicial o en $1/(n \cdot L_{nn})$).

Esto disminuye ligeramente el nivel de feromona en esa arista y favorece la exploración de rutas alternativas dentro de la misma iteración.

4. Actualización local de la feromona

Al finalizar una iteración, se aplica una actualización global usando la mejor ruta encontrada (de la iteración o la mejor histórica):

$$\tau_{ij}(t+1) = (1 - \rho) \tau_{ij}(t) + \rho \Delta \tau_{ij}^{best}$$

donde:

$$\Delta\tau_{ij}^{best} = \begin{cases} \frac{1}{L_{best}} & \text{si la arista } ij \text{ pertenece al recorrido mejor} \\ 0 & \text{en otro caso} \end{cases}$$

La evaporación (factor ρ) evita que las feromonas crezcan sin control y mantiene variedad en las rutas.

ρ : tasa de evaporación global ($0 < \rho < 1$).

L_{best} : longitud (costo) de la mejor ruta usada para depositar la feromona.

5. Final del código

El proceso continúa hasta cumplir una condición de paro. Lo más común es usar un **número fijo de iteraciones**, aunque también puede detenerse al alcanzar un recorrido suficientemente corto o cuando el sistema converge.

Implementación

Inicialmente importamos numpy, time y matplotlib, como sigue:

```
import numpy as np
from time import time
import matplotlib.pyplot as plt
```

Archivo TSP

Un archivo de un problema del viajante simétrico (TSP) tiene la siguiente estructura:

txt

- 1- NAME : <string>
- 2- TYPE : <string>
- 3- COMMENT : <string>
- 4- DIMENSION : <integer>
- 5- EDGE WEIGHT TYPE : <string>
- 6- NODE COORD SECTION : <integer> <real> <real>
- 7- EOF

1 — Identifica el archivo de datos.

2 — Especifica el tipo de datos (TSP: datos para un problema del viajante simétrico).

3 — Comentarios adicionales (habitualmente se indica el autor o contribuyente del caso).

4 — En un TSP, DIMENSION es el número de nodos.

5 — Especifica cómo se dan los pesos de las aristas (por ejemplo EUC_2D: distancias euclidianas en 2D).

6 — En esta sección se dan las coordenadas de los nodos; cada línea tiene la forma: id x y.

7 — Finaliza los datos de entrada (opcional).

Con la función `load_tsp_space` podemos llamar a un método que lea nuestro archivo TSP y almacene sus datos para usarlos después. Para comenzar utilizaremos el problema de 100 ciudades.

```
space = load_tsp_space(
    "data/kroA100.tsp"
)
```

A continuación, llamamos al algoritmo. Los parámetros del algoritmo serán:

{numpy.ndarray} space — El espacio (coordenadas).

{int} iterations {1000} — Número de iteraciones (condición de paro).

{int} colony {80} — Número de hormigas en la colonia.

{float} alpha {1.0} — Parámetro α : peso de la feromona.

{float} beta {5.0} — Parámetro β : peso de la heurística (visibilidad).

{float} del_tau {0.1} — delta_tau: tasa de depósito de feromona.

{float} rho {0.1} — rho: tasa de evaporación global de feromonas.

{float} xi {0.1} — Tasa de evaporación local de feromona usada en la actualización local.

Funciones del programa

Cálculo de la matriz de distancias

Calcula la distancia euclidiana entre todos los pares de nodos usando NumPy vectorizado.

```
def compute_distances(space):
    n = len(space)
    dist = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            dx = space[i][0] - space[j][0]
            dy = space[i][1] - space[j][1]
```

```

        dist[i][j] = int(round(np.sqrt(dx*dx + dy*dy)))
    return dist

```

Cálculo de probabilidades de transición

Aplica la fórmula del ACS usando feromona, visibilidad y parámetros α y β .

```

def formulaprob(pher, vis, alpha, beta, current, visited):
    nodos = [i for i in range(len(pher)) if i not in visited]
    numerador = np.array([
        (pher[current][j] ** alpha) * (vis[current][j] ** beta) for
j in nodos
    ])
    prob = numerador / numerador.sum()
    return nodos, prob

```

Actualización local de feromonas

Regla local del ACS para promover exploración.

```

def actualizacion_local(pher, i, j, xi, tau0):
    pher[i][j] = (1 - xi) * pher[i][j] + xi * tau0
    pher[j][i] = pher[i][j] # simétrico

```

Actualización global de feromonas

Evapora toda la matriz y refuerza solo la mejor ruta de la iteración.

```

def actualizacion_global(pher, ruta, costo, phi):
    pher *= (1 - phi)
    delta = phi * (1 / costo)
    for i in range(len(ruta) - 1):
        a, b = ruta[i], ruta[i+1]
        pher[a][b] += delta
        pher[b][a] += delta

```

Ejecución del ACO

Para cada una de las 33 corridas:

Se inicializa la matriz de feromonas con τ_0 .

Cada iteración construye rutas completas para todas las hormigas.

La mejor ruta de la iteración se usa para la actualización global.

Se mantiene un registro de la mejor solución global.

El algoritmo mide el tiempo de ejecución de cada corrida y almacena sus costos finales.

```

for c in range(corridas):
    pher = np.full((len(space), len(space)), tau0)
    best_cost = float("inf")
    best_route = None
    tiempos[c] = 0
    t0 = time()

    for i in range(iters):
        rutas = []
        costos = []

        for k in range(num_hormigas):
            start = np.random.randint(len(space))
            ruta = [start]
            visited = {start}
            costo = 0

            while len(ruta) < len(space):
                current = ruta[-1]
                nodos, prob = formulaprob(pher, vis, alpha, beta,
current, visited)
                siguiente = np.random.choice(nodos, p=prob)
                costo += dist[current][siguiente]
                ruta.append(siguiente)
                visited.add(siguiente)
                actualizacion_local(pher, current, siguiente, xi, tau0)

            costo += dist[ruta[-1]][ruta[0]]
            rutas.append(ruta)
            costos.append(costo)

        idx = np.argmin(costos)
        if costos[idx] < best_cost:
            best_cost = costos[idx]
            best_route = rutas[idx]

        actualizacion_global(pher, best_route, best_cost, phi)

    tiempos[c] = time() - t0
    costos_finales[c] = best_cost

```

Almacenamiento de resultados

Se guardan en archivos .npy:

tiempos totales por corrida,

costos finales por corrida,

la mejor ruta global entre todas las corridas.

Esto permite análisis posteriores sin necesidad de recomputar.

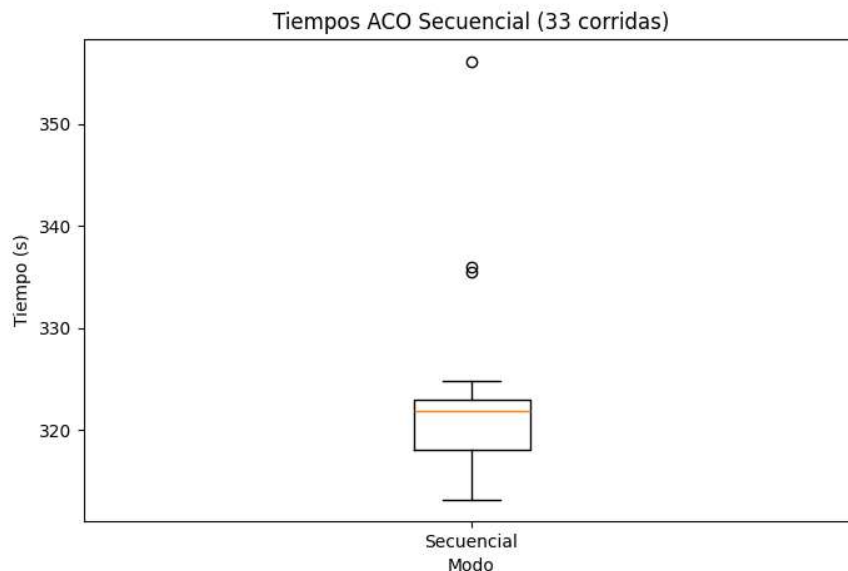
```
np.save("tiempos.npy", tiempos) np.save("costos.npy",  
costos_finales) np.save("mejor_ruta.npy", best_route)
```

Visualización

Finalmente, se genera un boxplot con los tiempos de las 33 corridas, permitiendo analizar la variabilidad temporal del algoritmo.

La figura se guarda automáticamente como archivo PNG.

```
plt.boxplot(tiempos)  
plt.ylabel("Tiempo (s)")  
plt.title("Distribución de tiempos por corrida")  
plt.savefig("tiempos_boxplot.png")  
plt.close()
```



Paralelización Basada en Procesos del ACO para el Problema del Viajero (TSP)

Esta implementación usa múltiples procesos para acelerar el algoritmo ACO. Cada proceso genera un subconjunto de hormigas de forma independiente, y el proceso principal integra los resultados después de cada iteración.

1. Distribución de hormigas por proceso

Cada proceso calcula cuántas hormigas debe construir:

```
hormigas_local = hormigas_totales // numero_procesos
```

Esto divide equitativamente el trabajo y garantiza que todos los procesos generen rutas completas.

2. Proceso trabajador (worker_aco)

Cada proceso ejecuta esta función. Dentro de ella se construyen todas las hormigas asignadas y se devuelve solo la mejor ruta encontrada localmente:

```
def worker_aco(i, numero_procesos, hormigas_totales, pher, dist,
eta, alpha, beta, xi, tau0):

    hormigas_local = hormigas_totales // numero_procesos

    N = pher.shape[0]

    best_cost = float("inf")

    best_route = None

    for _ in range(hormigas_local):

        visitadas = np.zeros(N, dtype=bool)

        nodo = np.random.randint(0, N)

        visitadas[nodo] = True

        ruta = [nodo]

        while not np.all(visitadas):

            nodos_disp, prob = formulaprob(alpha, beta, eta, pher,
visitadas, nodo)

            siguiente = np.random.choice(nodos_disp, p=prob)

            ruta.append(siguiente)

            actualizacion_local(pher, nodo, siguiente, xi, tau0)

            visitadas[siguiente] = True

            nodo = siguiente
```

```

        costo = sum(dist[ruta[i], ruta[(i + 1) % N]] for i in
range(N))

        if costo < best_cost:

            best_cost = costo

            best_route = ruta

    return best_route, best_cost

```

3. Creación del Pool de procesos

En cada corrida se crea un conjunto de procesos que trabajarán simultáneamente:

```
pool = mp.Pool(proc)
```

proc es la cantidad de procesos en nuestro caso usamos 8.

4. Ejecución paralela por iteración

En cada iteración del ACO, el código lanza proc tareas en paralelo:

```

results = [
    pool.apply_async(
        worker_aco,
        (i, proc, hormigas_totales, pher, dist, eta, alpha, beta,
xi, tau0)
    )
    for i in range(proc)
]

results = [r.get() for r in results]

```

Cada proceso construye sus hormigas y devuelve su mejor solución.

5. Selección del mejor resultado global en esa iteración

El proceso principal recopila los resultados y selecciona el mejor:

```
ruta_best, costo_best = min(results, key=lambda x: x[1])
```

Solo se usa una ruta por iteración para reforzar feromonas.

6. Actualización global centralizada

Una vez elegido el mejor resultado global de la iteración, el proceso principal ajusta la matriz de feromonas:

```
actualizacion_global(pher, ruta_best, costo_best, rho)
```

Los procesos no actualizan feromonas globales por sí mismos, evitando condiciones de carrera.

7. Medición de tiempos y repetición de corridas

Cada corrida se repite varias veces (33) y se recolectan tiempos:

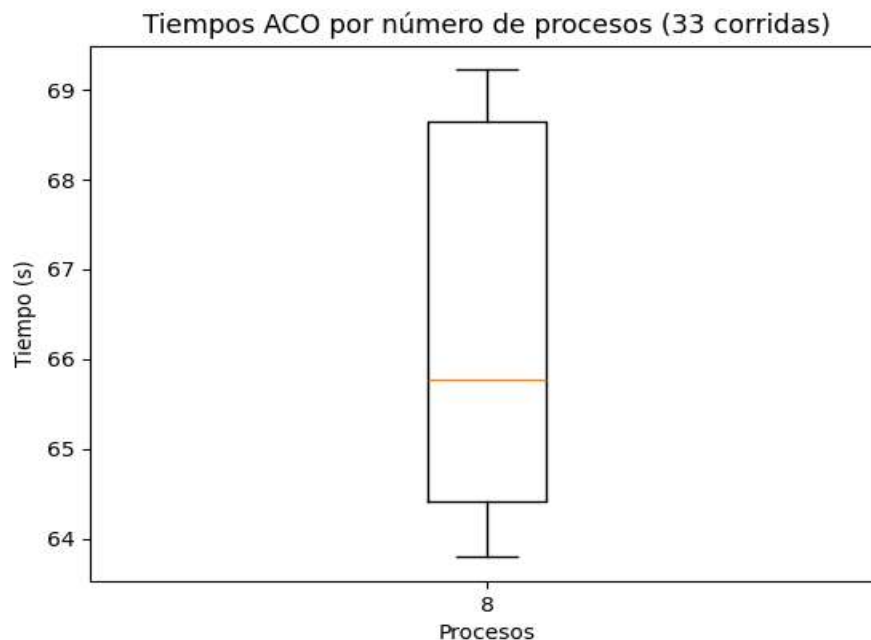
```
tiempos.append(time() - st)
resultados.append(tiempos[:])
```

Esto permite comparar el rendimiento del paralelismo.

8. Visualización del rendimiento

Finalmente se genera un boxplot para comparar los tiempos:

```
plt.boxplot(resultados, labels=[str(p) for p in procesos])
```



Algoritmo ACO Paralelo con MPI

Esta versión del algoritmo ACO distribuye el trabajo entre múltiples procesos usando MPI. Cada proceso construye un subconjunto de hormigas y el proceso raíz (rank 0) se encarga de reunir los resultados y actualizar las feromonas globales.

1. Distribución inicial de datos

Solo el proceso 0 carga el archivo TSP y calcula:

matriz de distancias (dist), matriz heurística (eta) y tamaño del problema(N).

```
if rank == 0:
    pace = load_tsp_space("MPI_DEBIAN_WSL/kroA100.tsp")
    N = space.shape[0]

    diff = space[:, None, :] - space[None, :, :]
    dist = np rint(np.sqrt(np.sum(diff**2, axis=2))).astype(int)
    eta = np.zeros_like(dist, dtype=float)
    mask = dist > 0
    eta[mask] = 1.0 / dist[mask]
else:
    N = None
    dist = None
    eta = None
```

Luego estos datos se distribuyen a todos los procesos con broadcast, todos los procesos comienzan con la misma información del TSP.

```
N = comm.bcast(N, root=0)
dist = comm.bcast(dist, root=0)
eta = comm.bcast(eta, root=0)
```

2. Asignación de hormigas por proceso

Cada proceso obtiene exactamente la misma cantidad de hormigas:

```
hormigas_local = n_hormigas // size
```

3. Cada proceso genera sus hormigas

Dentro de cada iteración, cada proceso construye sus propias rutas usando una copia local de las feromonas:

```
for it in range(iteraciones):
    mejor_local = float("inf")
    mejor_ruta_local = None
    pher_local = pher.copy()
```

```

for _ in range(hormigas_local):
    visitadas = np.zeros(N, dtype=bool)
    nodo = np.random.randint(0, N)
    visitadas[nodo] = True
    ruta = [nodo]

    while not np.all(visitadas):
        nodos_disp, prob = formulaprob(alpha, beta, eta,
pher_local, visitadas, nodo)
        siguiente = np.random.choice(nodos_disp, p=prob)
        actualizacion_local(pher_local, nodo, siguiente, xi,
tau0)
        ruta.append(siguiente)
        visitadas[siguiente] = True
        nodo = siguiente
    costo = sum(dist[ruta[i], ruta[(i + 1) % N]] for i in
range(N))
    if costo < mejor_local:
        mejor_local = costo
        mejor_ruta_local = ruta

```

Cada proceso:

genera hormigas, aplica sólo la actualización local y obtiene su mejor ruta local.

4. Recolección de mejores soluciones de cada proceso

Los resultados se juntan en el proceso 0 con gather:

```

best_costs = comm.gather(mejor_local, root=0)
best_routes = comm.gather(mejor_ruta_local, root=0)

```

Cada proceso envía:

el mejor costo local y la mejor ruta local.

5. Selección del mejor de la iteración (solo rank 0)

El proceso raíz selecciona el mejor resultado de todos:

```
idx = np.argmin(best_costs)
mejor_iter = best_costs[idx]
mejor_ruta_iter = best_routes[idx]
```

Y aplica la actualización global:

```
actualizacion_global(pher, mejor_ruta_iter, mejor_iter, rho)
```

6. Sincronización de feromonas

Después de actualizar las feromonas, rank 0 las transmite a todos:

```
pher = comm.bcast(pher, root=0)
```

Esto asegura que todos los procesos estén trabajando siempre con la misma matriz global al iniciar la siguiente iteración.

7. Repetición del proceso y registro de tiempos

Solo rank 0 mide tiempos:

```
if rank == 0:
    inicio = time()
```

Y al finalizar cada corrida:

```
resultados.append(duracion)
```

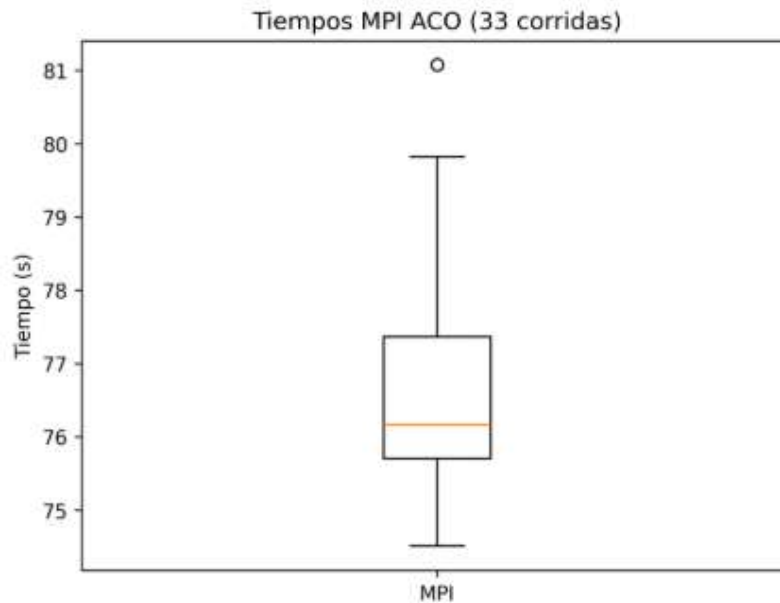
8. Guardado de resultados finales

Al terminar todas las corridas, rank 0 guarda:

```
np.save("MPI_final_pher2.npy", pher)
np.save("MPI_final_ruta2.npy", np.array(mejor_ruta_global))
np.save("MPI_final_costo2.npy", np.array([mejor_costo_global]))
np.save("MPI_final_tiempos2.npy", np.array(resultados))
```

Y genera un boxplot para medir rendimiento:

```
plt.boxplot(resultados, tick_labels=["MPI"])
```



Conclusiones:

El algoritmo ACO representa un reto interesante inspirado en la naturaleza, y aplicado al TSP se convierte en un problema aún más desafiante debido a la cantidad de decisiones y parámetros que intervienen. Implementarlo permitió comprender mejor su lógica interna y, sobre todo, aprender cómo llevarla al paralelismo tanto con procesos como con MPI, cada uno con características distintas para enfrentar el mismo problema.

Los resultados muestran mejoras claras cuando se dispone de más tiempo de cómputo y se ajustan adecuadamente los hiperparámetros, aunque también dependen del hardware, de la eficiencia en la programación y de condiciones externas que pueden parecer menores, como si la computadora está conectada a la corriente o trabajando con distintas cargas del sistema.

En conjunto, este trabajo evidenció que el paralelismo no solo acelera el ACO, sino que también abre la puerta a resolver instancias más grandes y complejas, siempre considerando las múltiples variables que influyen en su rendimiento.

Referencias:

mastqe. (s.f.). *TSPLIB solutions repository*. GitHub.

<https://github.com/mastqe/tsplib/blob/master/solutions>